BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Trần Lê Minh Tâm
MSSV / Lớp	3124720064/DNT1241

Mục tiêu học tập

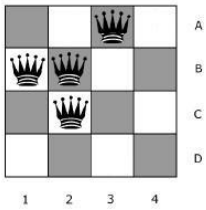
- Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).
- Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.
- Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.
- Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.
- Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Sau một vòng lặp (di chuyển một quân hậu), thuật toán sẽ chọn trạng thái lân cận (neighbor) sao cho tổng số lượng xung đột (tấn công nhau) giữa các quân hậu giảm xuống mức thấp nhất

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng

X B) Cực trị cục bộ (local optimum)

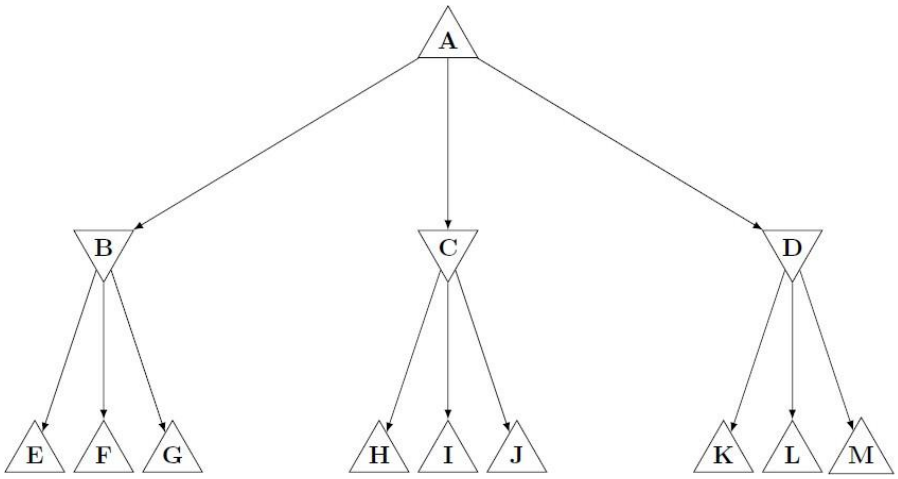
☐ C) Cực trị toàn cục (global optimum)

☐ D) Vai bằng (shoulder)

☐ E) Không thuộc các trường hợp trên

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D

☐ C → D → B

X D → C → B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

I, J (thuộc nhánh C) và **F, G** (thuộc nhánh B).

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

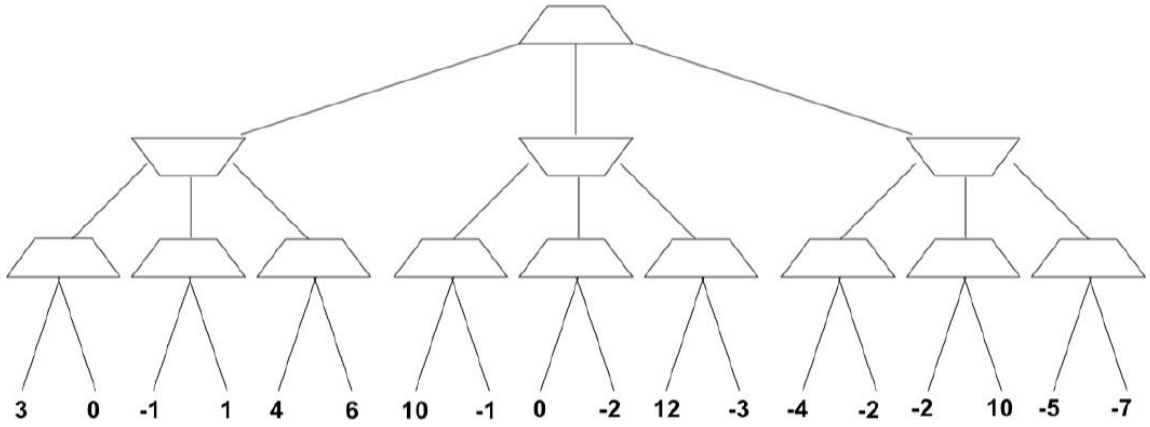
X Có

☐ Không

Việc sắp xếp thứ tự duyệt các nhánh dựa trên hàm đánh giá không làm thay đổi giá trị Minimax thực sự của cây. Nó chỉ giúp thuật toán nhanh chóng tìm thấy các giá trị α, β tốt hơn ở các nhánh đầu tiên, từ đó tăng số lượng nhánh bị cắt tỉa (pruning) ở các bước sau, giúp thuật toán chạy nhanh hơn.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

- Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.
- Ghi giá trị được trả về tại mỗi nút trung gian.
- Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.
- Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

Giá trị tại các nút:

Gốc (MAX): Nhận $\alpha = 1, \beta = +\infty$, trả về giá trị 1.

Nút MIN trái (MIN 1): Nhận $\alpha = -\infty, \beta = 1$, trả về giá trị 1.

Nút MIN giữa (MIN 2): Nhận $\alpha = 1, \beta = 0$, trả về giá trị 0.

Nút MIN phải (MIN 3): Nhận $\alpha = 1, \beta = -2$, trả về giá trị -2.

Các nhánh bị cắt tỉa

Tại nhánh MIN 1: Cắt nhánh lá có giá trị 6.

Tại nhánh MIN 2: Cắt toàn bộ nút MAX thứ 3 (chứa các lá 12, -3).

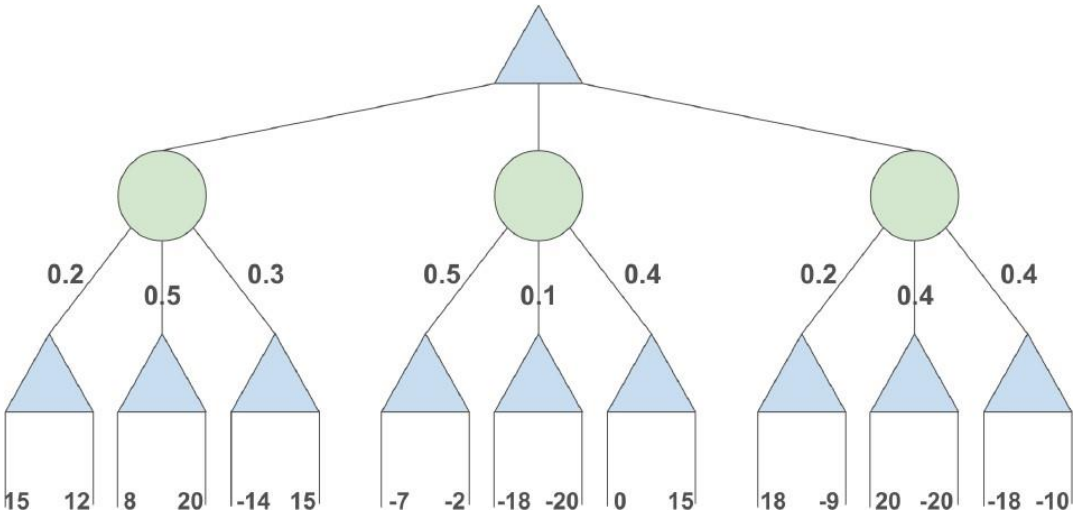
Tại nhánh MIN 3: Cắt toàn bộ nút MAX thứ 2 (chứa lá -2, 10) và nút MAX thứ 3 (chứa lá -5, -7).

Giá trị Minimax tại nút gốc: 1.

Nước đi tối ưu của MAX: Chọn nhánh bên trái (MIN 1).

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Nút Chance trái (C1): $(0.2 \times \max(15,12)) + (0.5 \times \max(8,20)) + (0.3 \times \max(-14,15)) = 3 + 10 + 4.5 = 17.5$

Nút Chance giữa (C2): $(0.5 \times \max(-7,-2)) + (0.1 \times \max(-18,-20)) + (0.4 \times \max(0,15)) = -1 - 1.8 + 6 = 3.2$

Nút Chance phải (C3): $(0.2 \times \max(18,-9)) + (0.4 \times \max(20,-20)) + (0.4 \times \max(-18,-10)) = 3.6 + 8 - 4 = 7.6$

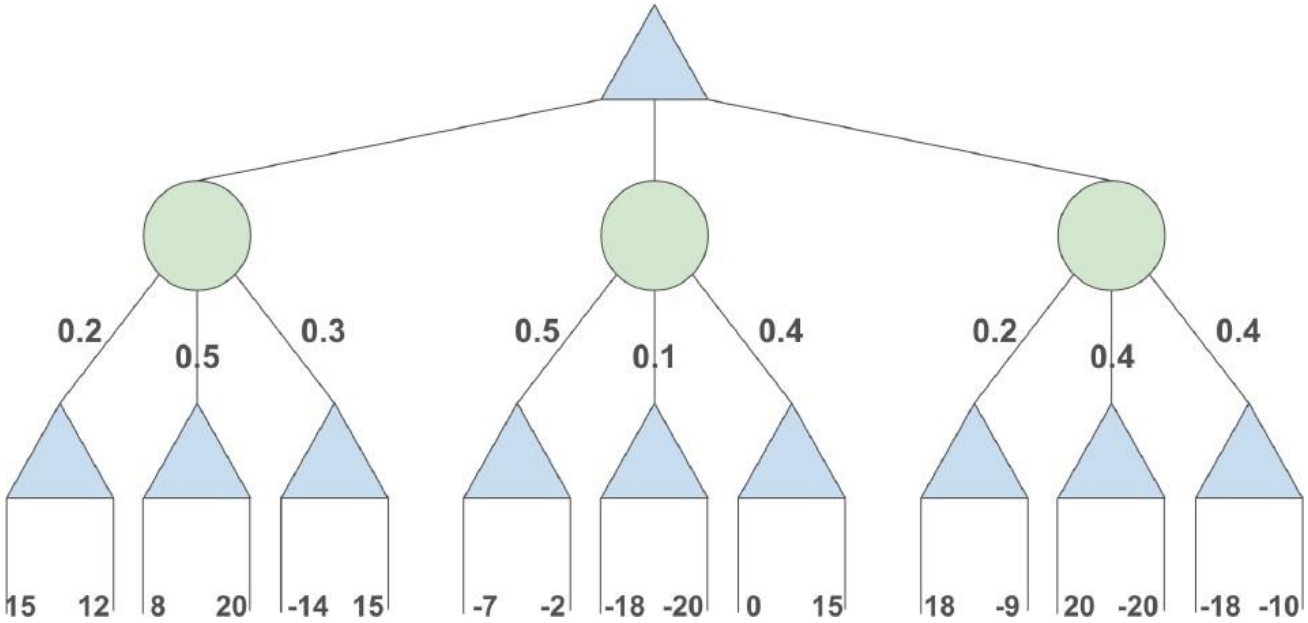
Nút gốc (MAX): $\max(17.5,3.2,7.6) = 17.5$

b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Không có nút nào có thể được cắt tỉa

Trong Expectimax, giá trị của một Chance node phụ thuộc vào trung bình cộng của mọi nhánh con. Nếu không có giới hạn, một nút lá chưa duyệt có thể mang giá trị vô cùng lớn hoặc vô cùng nhỏ, làm thay đổi hoàn toàn giá trị kỳ vọng của Chance node đó.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Có thể cắt tỉa hai nút MAX cuối của nhánh Chance giữa (C2) cùng các lá của chúng. Khi nhánh con đầu tiên của C2 trả về -2, dù hai nhánh còn lại có đạt giá trị tối đa là 20, kỳ vọng lớn nhất có thể của C2 chỉ là $(0.5 \times -2) + (0.1 \times 20) + (0.4 \times 20) = 9$. Vì $9 <$ 17.5 (giá trị đã có ở C1), gốc MAX chắc chắn không chọn C2 nên ta không cần duyệt tiếp nhánh này.

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Phạm vi lan truyền ràng buộc: Forward Checking (Kiểm tra tiến) có phạm vi lan truyền hẹp; nó chỉ duy trì tính nhất quán cung từ biến vừa được gán giá trị đến các biến kề chưa được gán với nó. AC-3 có phạm vi lan truyền rộng và sâu hơn; nó duy trì tính nhất quán cung trên toàn bộ mạng lưới các biến (toàn bộ đồ thị ràng buộc).

Thời điểm thực hiện: Forward Checking được kích hoạt ngay sau mỗi lần thuật toán thực hiện một phép gán giá trị cho một biến. AC-3 có thể được thực hiện như một bước tiền xử lý độc lập trước khi bắt đầu tìm kiếm, hoặc được gọi liên tục sau mỗi phép gán (thuật toán MAC - Maintaining Arc Consistency).

Chi phí tính toán: Forward Checking có chi phí tính toán tại mỗi bước (mỗi nút) thấp hơn do kiểm tra ít cung hơn, nhưng vì cắt tỉa yếu nên có thể để lại không gian tìm kiếm lớn và phải quay lui (backtrack) nhiều. AC-3 tốn nhiều chi

phí tính toán hơn tại mỗi bước, nhưng bù lại nó cắt tỉa triệt để các miền giá trị vô nghiệm, giúp giảm đáng kể số lần quay lui và tổng số nút phải duyệt.

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV)

Heuristic này ưu tiên chọn biến có số lượng giá trị hợp lệ còn lại trong miền ít nhất. Nó hoạt động dựa trên nguyên lý "fail-first" (ưu tiên thất bại sớm). Bằng cách xử lý phần khó nhất trước, MRV giúp thuật toán nhanh chóng phát hiện ra các nhánh chắc chắn dẫn đến vô nghiệm (dead-end) để tiến hành quay lui sớm, tránh việc lãng phí thời gian duyệt sâu vào các nhánh sai lệch.

ii) Least Constraining Value (LCV)

Thường được dùng dựa trên nguyên lý "Succeed-first" (Ưu tiên thành công). Khi đã chọn được biến cần gán, việc ưu tiên chọn giá trị ít gây cản trở (loại bỏ ít lựa chọn nhất) cho các biến kề chưa được gán sẽ giúp bảo toàn tối đa không gian lựa chọn cho hệ thống, từ đó làm tăng xác suất đi thẳng đến nghiệm mà không cần phải quay lui.

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

- Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.
- Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.
- Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.
- Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C.
- Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng.

a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

Biến: A, B, C, D, E.

Miền giá trị (Domain): $\{1Q, 2Q, 3Q, 4Q, 1N, 2N, 3N, 4N\}$.

Ràng buộc đơn: $B = 1, D \geq 3, A \leq 2$.

Ràng buộc nhị phân: $D < C$, và $AllDiff(A, B, C, D, E)$.

Bổ sung hai ràng buộc sau:

- A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.
- D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

Miền mới: $\{1,2,3,4\}$ (Vì nhóm A, B, C dùng băng Q và D, E dùng băng N đã cố định).

Đồ thị ràng buộc nhị phân: Cạnh $(A, B), (B, C), (C, A)$ do cùng băng Q nên phải khác khung giờ. Cạnh (D, E) do cùng băng N nên phải khác khung giờ. Cạnh có hướng $D \rightarrow C$ do ràng buộc $D < C$.

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

Từ ràng buộc đơn: $B = \{1\}, D = \{3,4\}, A = \{1,2\}$.

Lan truyền nhị phân: $A \neq B \Rightarrow A = \{2\}$. $C \neq B, C \neq A \Rightarrow C = \{3,4\}$. $D < C \Rightarrow D = \{3\}, C = \{4\}$. $E \neq D \Rightarrow E = \{1,2,4\}$.

Miền còn lại: **A={2}, B={1}, C={4}, D={3}, E={1, 2, 4}**.

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

B = 1

A = 2

C = 3 (Gây ra dead-end vì D không còn giá trị thỏa mãn $D < 3$, bị hoàn tác)

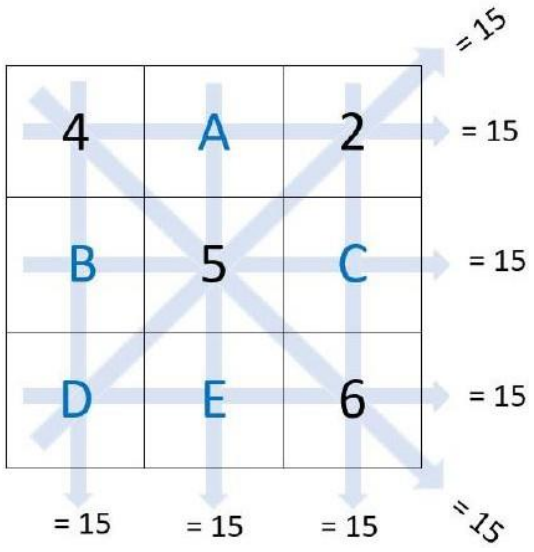
C = 4

D = 3

E = 1

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n²}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

Biến A, B, C, D, E với miền giá trị {1,3,7,8,9}.

Ràng buộc: $A = 9$, $B + C = 10$, $D + E = 9$, $B + D = 11$, $A + E = 10$, $C = 7$, $D = 8$, kèm theo $AllDiff(\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{E})$.

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.

Gồm 5 đỉnh liên kết với nhau qua các cạnh tương ứng với các phương trình 2 biến: $(B, C), (D, E), (B, D), (A, E)$.

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

Áp dụng các phương trình 1 biến ta có $A = 9, C = 7, D = 8$. Lan truyền qua các cạnh đồ thị nhị phân: $B + C = 10 \Rightarrow B = 3$; $D + E = 9 \Rightarrow E = 1$. Tập nghiệm cuối cùng là: A=9, B=3, C=7, D=8, E=1

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

- AJ = {Đỏ, Vàng, Xanh lá}
- Sunny = {Đỏ, Vàng, Hồng}
- Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

Thứ tự gán: Ellyse = Hồng → Sunny = Đỏ → AJ = Vàng.

Số lần backtrack: 0 lần.

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

Số lần backtrack: 1 lần (Gán AJ=Đỏ, Sunny=Hồng khiến Ellyse hết miền giá trị, phải backtrack gán lại Sunny=Vàng).

So sánh và giải thích: Số lần backtrack cao hơn câu (a). Việc dùng Maximum Remaining Values và Most Constraining Value đi ngược lại với triết lý tối ưu Heuristic của CSP. Chọn biến dễ (nhiều lựa chọn nhất) làm chậm quá trình phát hiện mâu thuẫn, và chọn giá trị khắt khe nhất (cản trở nhiều nhất) lại cố tình thu hẹp miền giá trị của các biến khác, dẫn đến bế tắc và buộc thuật toán phải quay lui.

--- Hết ---